GoBall - System Architecture Document

Mini Golf Scoring System for Raspberry Pi 5

Ahmed Abdelghani

February 2026

Contents

1 Introduction

1.1 Purpose

GoBall is a Raspberry Pi 5-based mini golf scoring system that combines hardware sensors, LED feedback, audio announcements, and a touchscreen UI into an interactive scoring experience. This document describes the system architecture, module interactions, hardware interfaces, and data flow.

1.2 Scope

This document covers:

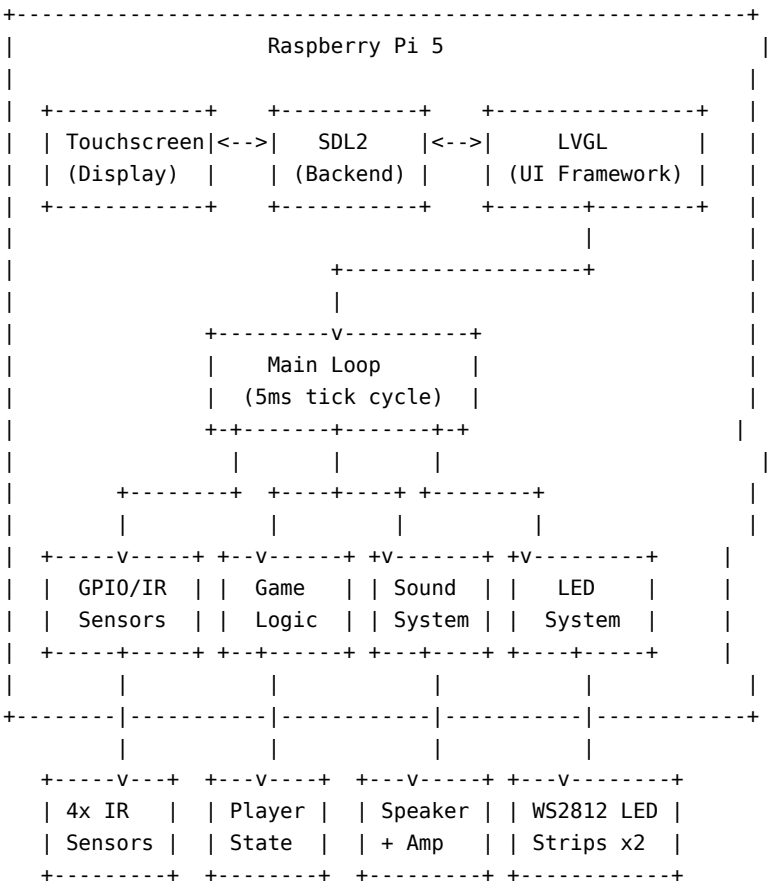
- Hardware components and wiring
- Software module architecture
- Game mode logic and state management
- Sensor input pipeline
- Audio and LED subsystems
- UI framework and screen management
- Build and deployment infrastructure

1.3 Definitions

Term	Definition
LVGL	Light and Versatile Graphics Library - embedded UI framework
PIO	Programmable I/O - RPi5 hardware for driving LED protocols
WS2812	Addressable RGB LED protocol (NeoPixel)
libgpiod	Linux GPIO character device library
SDL2	Simple DirectMedia Layer - display/audio backend
SquareLine Studio	Visual LVGL UI design tool

2 System Overview

2.1 High-Level Architecture

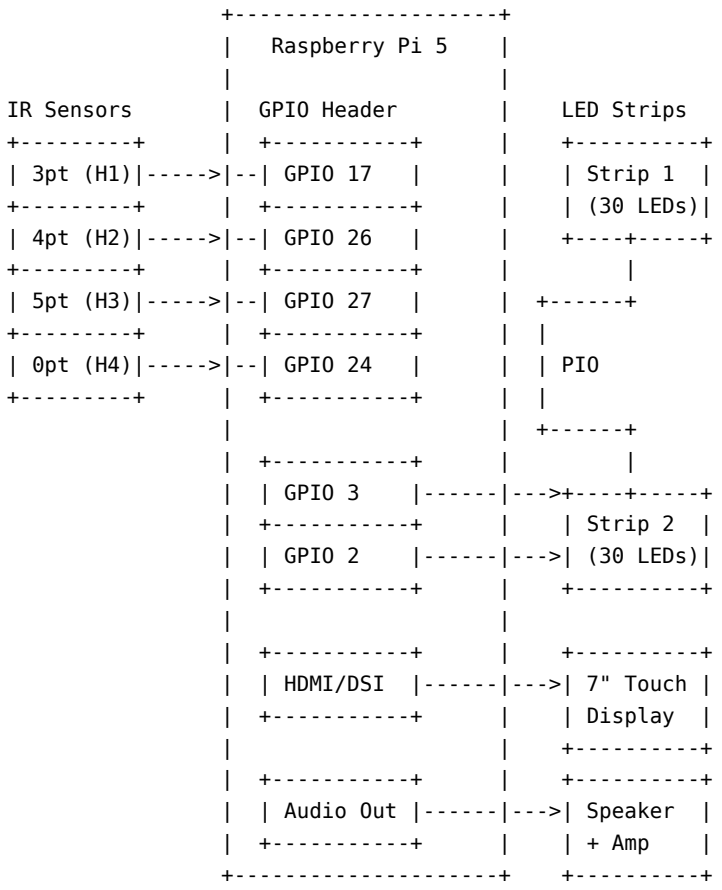


2.2 Component Summary

Component	Technology	Purpose
Display	7” touchscreen via SDL2/DRM	User interface rendering
UI Framework	LVGL v9 + SquareLine Studio	Screen layout, widgets, events
Game Logic	Custom C modules	Scoring rules, turn management
Sensors	libgpiod + GPIO interrupts	Ball detection (4 scoring holes)
Audio	SDL2_mixer	Voice announcements, sound effects
LEDs	PIO + WS2812 protocol	Color-coded scoring feedback
Build	CMake + aarch64 cross-compiler	Cross-compilation for RPi5

3 Hardware Architecture

3.1 Block Diagram



3.2 GPIO Pin Assignment

GPIO Pin	Function	Signal Type	Direction
17	3-Point Hole Sensor	Falling edge interrupt	Input
26	4-Point Hole Sensor	Falling edge interrupt	Input
27	5-Point Hole Sensor	Falling edge interrupt	Input
24	0-Point Hole Sensor	Falling edge interrupt	Input
3	LED Strip 1 Data	PIO-driven WS2812	Output
2	LED Strip 2 Data	PIO-driven WS2812	Output

3.3 IR Sensor Specifications

- Type: Active IR break-beam sensors
- Detection: Falling edge (beam broken = ball passed)
- Debounce: 3000ms signal-based POSIX timer per sensor
- Interface: `/dev/gpiochip0` via libgpiod

3.4 LED Strip Specifications

- Type: WS2812 (NeoPixel) addressable RGB+W

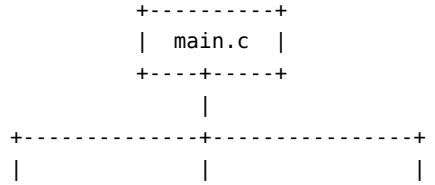
- Pixel Count: 30 per strip (configurable via `PIXELS` define)
- Data Rate: 800 kHz
- Interface: PIO (Programmable I/O) hardware on RPi5
- Color Order: WBGR (White, Blue, Green, Red)
- Brightness: Software-controllable 0-255

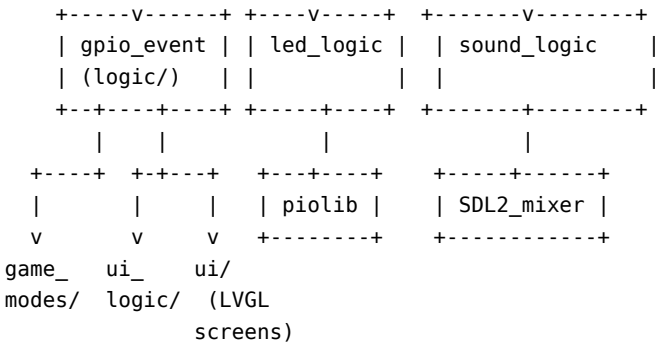
4 Software Architecture

4.1 Module Structure

```
SquareLine_Project/
|
+-- main.c                          Entry point, main loop
|
+-- modules/
|   +-- debug/                      Debug logging subsystem
|   |   +-- debug.h/c               Configurable per-module logging
|   |
|   +-- game_modes/                 Game rule implementations
|   |   +-- game_modes.h            GameMode, HoleMode, MatchPlayMode enums
|   |   +-- player.h                Player struct definition
|   |   +-- strokeplay.h/c          Stroke Play scoring
|   |   +-- matchplay.h/c           Match Play scoring
|   |   +-- quotaplay.h/c           Quota + Vegas Quota scoring
|   |
|   +-- logic/                      Core game flow controller
|   |   +-- gpio_event.h/c          GPIO init, event loop, turn management,
|   |                                scorecard updates, player highlights
|   |
|   +-- sound_logic/                Audio subsystem
|   |   +-- sound_logic_event.h/c   SDL2_mixer init, WAV loading,
|   |                                timed playback, mute control
|   |
|   +-- led_logic/                  LED strip subsystem
|   |   +-- led_logic_event.h/c     PIO init, animations, flash effects
|   |
|   +-- ui_logic/                   UI event handlers
|       +-- ui_logic.event.h/c      Button callbacks, screen transitions,
|                                    game mode setup from UI selections
|
+-- ui/                             SquareLine Studio generated UI
|   +-- ui.h/c                      UI globals, screen init, event wiring
|   +-- screens/                    All screen definitions
|   +-- images/                     Image assets (C arrays)
|   +-- fonts/                      Font assets (C arrays)
|   +-- components/                 Reusable UI components
|
+-- utils/
|   +-- piolib/                     PIO library for WS2812 control
|   +-- autostart/                  Kiosk mode desktop files
|
+-- lvgl/                           LVGL library source (v9)
```

4.2 Module Dependency Graph





4.3 Main Loop

The application runs a continuous loop at approximately 200 Hz (5ms sleep):

```
while (1)
{
    update_led_animation(&leds);        // Step 1: Update LED train animation
    logic_handle_events(...);           // Step 2: Poll GPIO, process scoring
    lv_timer_handler();                 // Step 3: LVGL tick (UI render + input)
    usleep(5000);                       // Step 4: 5ms sleep
}
```

Step 1 advances the LED train animation by one pixel position each frame.

Step 2 performs a non-blocking poll of GPIO lines. If a sensor fires, the full scoring chain executes synchronously: debounce check, scoring function call, detection count increment, UI label update, completion check, and turn advancement.

Step 3 runs LVGL's timer handler, which processes pending UI events, renders dirty regions, and handles SDL input (touch, mouse).

5 Sensor Input Pipeline

5.1 Event Flow

```
IR Beam Broken
      |
      v
GPIO Falling Edge (via libgpiod)
      |
      v
gpiod_line_event_wait_bulk()  [non-blocking]
      |
      v
sensors_enabled check  ----NO----> Ignore
      |
     YES
      v
pin_to_sensor_index(pin_offset)
      |
      v
Debounce Check (3s timer)  ----ACTIVE----> Ignore
      |
    CLEAR
      v
start_debounce_timer()
      |
      v
Game Mode Switch
      |
      +-> Stroke Play:  stroke_play_process_pin()
      +-> Match Play:   inline scoring (3/4/5/0 pts)
      +-> Quota:        quota_play_process_pin()
      +-> Vegas:        vegas_quota_play_process_pin()
      |
      v
detection_count++
      |
      v
logic_update_label_text()  [update ball counter UI]
      |
      v
Quota/Vegas: Check completion (par3==0 && par4==0 && par5==0)
      |
      v
detection_count == SENSORS_PER_TURN (2)?
      |
     YES
      v
Turn Completion Logic
      |
      +-> Stroke Play: update_scoreCard(), advance hole
      +-> Match Play:  evaluate hole winner, update Up&Down
      +-> Quota/Vegas: advance hole, announce next player
      |
```

```
      v
current_player_index = (current_player_index + 1) % num_players
      |
      v
check_all_players_completed()  [end-of-game detection]
```

5.2 Debounce Mechanism

Each sensor has an independent POSIX real-time timer using `SIGRTMIN` signals:

1. On first trigger: `sensor_debouncing[index] = true`, timer starts (3000ms)
2. During debounce window: all subsequent triggers on that sensor are ignored
3. On timer expiry: signal handler sets `sensor_debouncing[index] = false`
4. Each sensor debounces independently (different holes can trigger simultaneously)

5.3 Turn Structure

Each player gets **2 ball detections per turn** (`SENSORS_PER_TURN = 2`). After 2 detections:

- Scores are finalized for the turn
- Scorecard is updated
- Player index advances to the next player
- Player turn voice announcement plays

Exception: Quota/Vegas 1-player mode uses 1 detection per turn.

6 Game Modes

6.1 Stroke Play

Parameter	Value
Players	1-4
Holes	9 or 18
Scoring	Cumulative points per hole
Win Condition	Highest total score after all holes

Scoring: Each ball detection adds points based on which hole it enters (3, 4, 5, or 0 points). Two detections per turn are summed as the hole score.

Turn Cycling:

- 2 players: alternates every 8 detections (4 turns each)
- 3 players: cycles every 12 detections
- 4 players: cycles every 24 detections

6.2 Match Play

Parameter	Value
Mode	1v1 or 2v2
Holes	9 or 18
Scoring	Up & Down per hole
Win Condition	Lead exceeds remaining holes (early victory) or most holes won

Scoring: Both players/teams play the same hole. The higher scorer wins the hole. Scores are displayed as "XUP" / "XDN" / "E" (Even).

Early Victory: If a player's lead exceeds the number of remaining holes, the match ends immediately.

6.3 Quota Points

Parameter	Value
Players	1-4
Holes	9 or 18
Scoring	Par countdown (3pt, 4pt, 5pt targets)
Win Condition	First to complete all three par categories (reach 0,0,0)

Scoring: Each player starts with preset par targets (e.g., par3=3, par4=2, par5=1). Hitting a hole decrements the corresponding counter. The game ends when a player reaches 0 in all three categories.

6.4 Vegas Quota Points

Parameter	Value
Players	1-4
Holes	9 or 18
Scoring	Par countdown + bonus points
Win Condition	First to complete; ties broken by bonus score

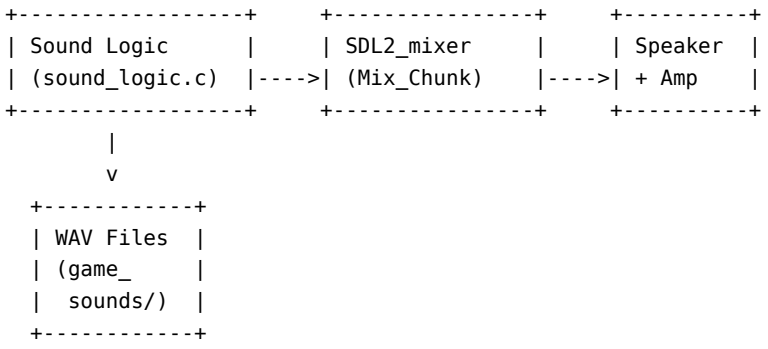
Scoring: Same countdown as Quota Points, but after a player completes a category (reaches 0), subsequent hits in that category earn bonus points.

Dead Category Rule: In multiplayer (2+), a category becomes "dead" (no bonus scoring) once 2 players have completed it. This prevents runaway bonus accumulation.

```
Category Lifecycle:
  OPEN ──(player completes)──> OPEN (1 closed)
       ──(2nd player completes)──> DEAD (no more bonus for anyone)
```

7 Audio Subsystem

7.1 Architecture



7.2 Sound Categories

Category	Examples	Trigger
Scoring	3points.wav, 4points.wav, 5points.wav, 0points.wav	Ball detection
Player Turn	playerone.wav through playerfour.wav	Turn switch
Team Turn	teamone.wav, teamtwo.wav	Match Play turn switch
Winner	p1wins1-5.wav, t1wins1-5.wav	Game completion
Navigation	mainmenu.wav, goingback.wav	Button press

7.3 Timed Playback

Sound playback uses LVGL timers for non-blocking operation:

```
play_sound_once(Mix_Chunk* sound, uint32_t delay_ms);
```

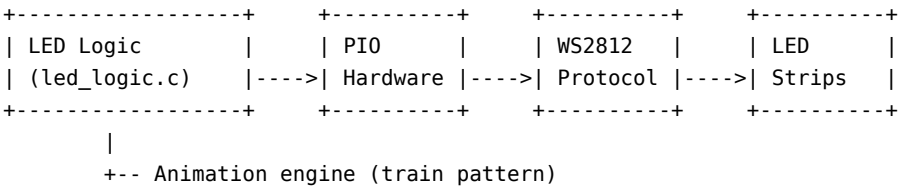
This creates a one-shot LVGL timer that plays the sound after the specified delay. This prevents audio from blocking the main loop and allows staggering of sequential announcements.

7.4 Configurable Delays

Define	Default	Purpose
`SOUND_DELAY_PLAYER_ANNOUNCE_MS`	750ms	Player turn announcement
`SOUND_DELAY_TEAM_ANNOUNCE_MS`	750ms	Team turn announcement
`SOUND_DELAY_PLAYER_WINS_MS`	750ms	Winner announcement
`SOUND_DELAY_TURN_SWITCH_MS`	500ms	Turn switch announcement

8 LED Subsystem

8.1 Architecture



8.2 Color Format

Colors use WBGR order (White, Blue, Green, Red) as `wbgr_color_t`:

Constant	W	B	G	R	Visual
`COLOR_RED`	0x00	0x00	0x00	0xFF	Red
`COLOR_GREEN`	0x00	0x00	0xFF	0x00	Green
`COLOR_BLUE`	0x00	0xFF	0x00	0x00	Blue
`COLOR_WHITE`	0xFF	0xFF	0xFF	0xFF	White

8.3 LED Behaviors

Idle Animation: A "train" pattern of colored pixels travels along the strip. Multiple trains (defined by `NUM_TRAINS`) are evenly spaced and advance one pixel per main loop iteration.

Scoring Flash: On ball detection, the animation pauses and LEDs flash:

- **Points scored (3/4/5):** Green flash for 1000ms
- **Zero points:** White flash for 1000ms
- Flash toggles on/off at 50ms intervals
- Animation automatically resumes after flash duration

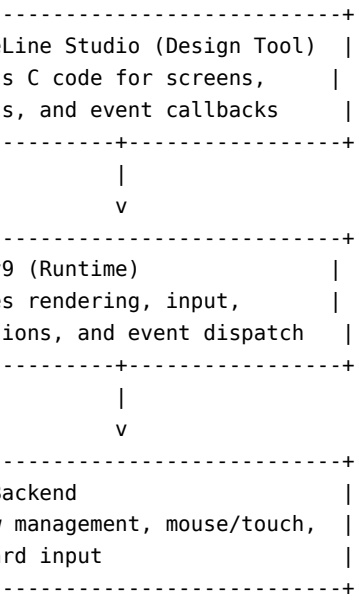
8.4 Graceful Degradation

The LED subsystem initializes with graceful fallback:

1. `pio_open(0)` - attempt to open PIO hardware
2. If PIO unavailable (e.g., desktop/QEMU): `controller.enabled = false`
3. If state machines unavailable: `controller.enabled = false`
4. All LED functions check `controller->enabled` before hardware access

9 UI Architecture

9.1 Framework Stack



9.2 Screen Hierarchy

The UI contains screens for each game mode variant:

```
Main Menu
|
+-- Stroke Play
|   +-- Player Select (1P/2P/3P/4P)
|   +-- Hole Select (9H/18H)
|   +-- Game Screen (per variant: SP1P9H, SP2P18H, etc.)
|   +-- Scorecard Screen
|
+-- Match Play
|   +-- Mode Select (1v1/2v2)
|   +-- Hole Select (9H/18H)
|   +-- Game Screen (MP1V19H, MP2V218H, etc.)
|   +-- Scorecard Screen
|
+-- Quota Points
|   +-- Player Select (1P/2P/3P/4P)
|   +-- Hole Select (9H/18H)
|   +-- Game Screen (Q1P9H, Q4P18H, etc.)
|
+-- Vegas Quota Points
    +-- Player Select (1P/2P/3P/4P)
    +-- Hole Select (9H/18H)
    +-- Game Screen (VQ1P9H, VQ4P18H, etc.)
```

9.3 UI Widget Naming Convention

SquareLine Studio generates widget names following this pattern:

`ui_[Mode][Players][Holes]GS[Widget][Detail]`

Examples:

Widget Name	Meaning
`ui_SP2P9HGSBCPText`	Stroke Play, 2-Player, 9-Hole, Game Screen, Ball Counter, Player Text
`ui_MP1V19HScP1SText3`	Match Play, 1v1, 9-Hole, Scorecard, Player 1, Score Text, Hole 3
`ui_VQ3P18HGSBSText`	Vegas Quota, 3-Player, 18-Hole, Game Screen, Bonus Score Text
`ui_Q4P9HGSMMButton`	Quota, 4-Player, 9-Hole, Game Screen, Main Menu Button

9.4 Event Handling

UI events are wired in two layers:

1. **SquareLine-generated callbacks** (`ui/ui.c`): Button presses trigger navigation and sound macros
2. **Custom logic callbacks** (`modules/ui_logic/ui_logic.event.c`): Game setup, score display, player highlighting, screen transitions on game completion

10 State Management

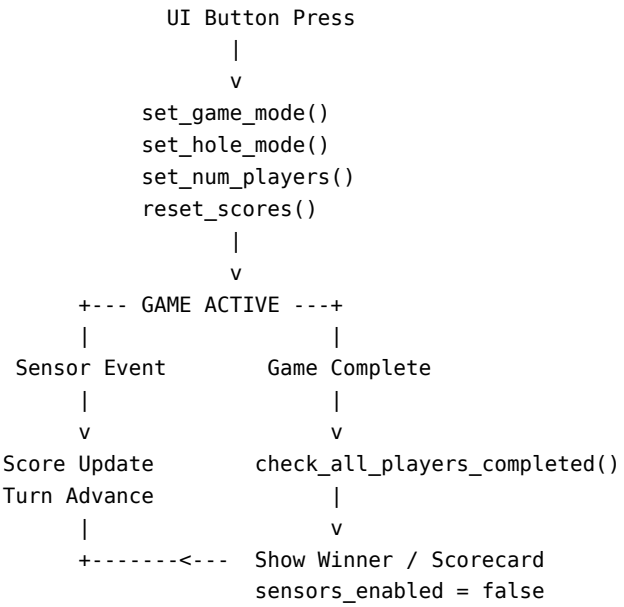
10.1 Global Game State

Variable	Type	Purpose
`current_game_mode`	`GameMode`	Active game mode enum
`current_hole_mode`	`HoleMode`	9 or 18 holes
`current_match_play_mode`	`MatchPlayMode`	1v1 or 2v2
`players[]`	`Player[4]`	Player state array
`current_player_index`	`int`	Whose turn it is
`num_players`	`int`	Active player count
`sensors_enabled`	`bool`	Sensor input gate
`update_flag`	`uint8_t`	Game completion flag

10.2 Player Struct

```c
typedef struct {
    uint8_t score;              // Cumulative score (Stroke/Quota)
    uint8_t current_hole;       // Current hole number
    uint8_t detection_count;    // Detections in current turn
    uint8_t round_total_score;  // Per-round score (Match Play)
    int8_t  upAndDown;          // Match Play standing
    uint8_t holes_won;          // Match Play holes won
    uint8_t par3_count;         // Quota: remaining 3pt pars
    uint8_t par4_count;         // Quota: remaining 4pt pars
    uint8_t par5_count;         // Quota: remaining 5pt pars
} Player;
```

10.3 State Transitions



11 Debug System

11.1 Architecture

The debug system provides configurable per-module logging with 5 severity levels:

Level	Value	Macro	Use Case
ERROR	1	`DEBUG_ERROR()`	Fatal/unrecoverable issues
WARN	2	`DEBUG_WARN()`	Unexpected but handled conditions
INFO	3	`DEBUG_INFO()`	Significant operational events
DEBUG	4	`DEBUG_DEBUG()`	Detailed diagnostic information
TRACE	5	`DEBUG_TRACE()`	Function entry/exit, per-frame data

11.2 Build-Type Mapping

Build Type	Debug Level	Defines
Debug	4 (DEBUG)	`DEBUG=1 DEBUG_LEVEL=4`
Debug + Trace	5 (TRACE)	`DEBUG=1 DEBUG_LEVEL=5 ENABLE_DEBUG_TRACE`
Release	2 (WARN)	`DEBUG_LEVEL=2`

11.3 Module Tags

Each log line is tagged with a module identifier for filtering:

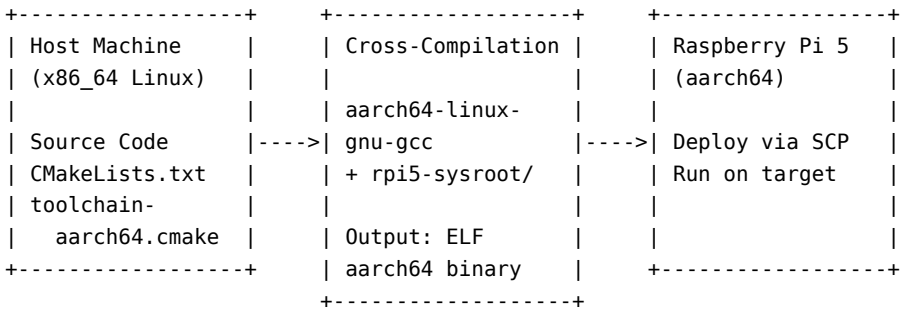
`MODULE_MAIN`, `MODULE_LVGL`, `MODULE_GPIO`, `MODULE_GAME`, `MODULE_SOUND`, `MODULE_LED`, `MODULE_UI`, `MODULE_HAL`, `MODULE_LOGIC`

11.4 Features

- **Colored output:** ANSI color codes per severity level (configurable via `ENABLE_DEBUG_COLORS`)
- **Timestamps:** Millisecond-precision timestamps (configurable via `ENABLE_DEBUG_TIMESTAMP`)
- **Zero overhead in Release:** Macros compile to no-ops when level is below threshold

12 Build and Deployment

12.1 Build Pipeline



12.2 CMake Configuration

```
# Standard build
cmake -B build -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
make -C build -j$(nproc)

# Release build
cmake -B build -DCMAKE_BUILD_TYPE=Release

# With trace logging
cmake -B build -DENABLE_DEBUG_TRACE=ON

# Custom sysroot
cmake -B build -DSYSROOT_PATH=/path/to/rpi5-sysroot
```

12.3 Build Options

Option	Default	Description
`CMAKE_BUILD_TYPE`	Debug	Debug (level 4) or Release (level 2)
`ENABLE_DEBUG_TRACE`	OFF	Enable TRACE-level logging
`ENABLE_DEBUG_COLORS`	ON	Colored terminal output
`ENABLE_DEBUG_TIMESTAMP`	ON	Timestamps in log output
`SYSROOT_PATH`	`rpi5-sysroot/`	RPi5 filesystem sysroot
`SOUND_DIR_PATH`	–	Override sound file directory
`YOCTO_BUILD`	–	Skip local toolchain paths

12.4 Dependencies

12.4.1 Host (Build Machine)

Package	Purpose
`cmake` >= 3.15	Build system
`aarch64-linux-gnu-gcc`	Cross-compiler
`pkg-config`	Library discovery
`rpi5-sysroot/`	Target filesystem headers/libs

12.4.2 Target (Raspberry Pi 5)

Package	Purpose
`libsdl2-2.0-0`	Display backend
`libsdl2-mixer-2.0-0`	Audio playback
`libgpiod2`	GPIO access

12.5 Deployment

```
# Copy binary to Pi
scp build/SquareLine_Project q@<rpi-ip>:~/Desktop/SquareLine_Project/

# Run on Pi
ssh q@<rpi-ip> './Desktop/SquareLine_Project/SquareLine_Project'
```

13 Appendix A: File Reference

File	Lines	Purpose
`main.c`	~175	Application entry, main loop, HAL init
`modules/logic/gpio_event.c`	~4100	GPIO init, scoring chain, turn management, scorecard
`modules/logic/gpio_event.h`	~97	Pin definitions, game state externs, function prototypes
`modules/ui_logic/ui_logic.event.c`	~6500	UI callbacks, score reset, scorecard updates, highlights
`modules/game_modes/strokeplay.c`	~50	Stroke play per-pin scoring
`modules/game_modes/matchplay.c`	~50	Match play per-pin scoring
`modules/game_modes/quotaplay.c`	~195	Quota + Vegas quota scoring with dead-category rule
`modules/game_modes/player.h`	~30	Player struct definition
`modules/game_modes/game_modes.h`	~30	GameMode, HoleMode, MatchPlayMode enums
`modules/sound_logic/sound_logic_event.c`	~200	Audio init, WAV loading, timed playback
`modules/led_logic/led_logic_event.c`	~215	PIO init, LED animation, flash effects
`modules/debug/debug.c`	~100	Debug init, level config, formatted output
`ui/ui.c`	~7400	SquareLine-generated screens and event wiring

14 Appendix B: Enum Definitions

```c
typedef enum {
    GAME_MODE_STROKE_PLAY,
    GAME_MODE_MATCH_PLAY,
    GAME_MODE_QUOTA,
    GAME_MODE_VEGAS
} GameMode;

typedef enum {
    NINE_HOLES = 9,
    EIGHTEEN_HOLES = 18
} HoleMode;

typedef enum {
    MATCH_PLAY_MODE_1V1,
    MATCH_PLAY_MODE_2V2
} MatchPlayMode;
```